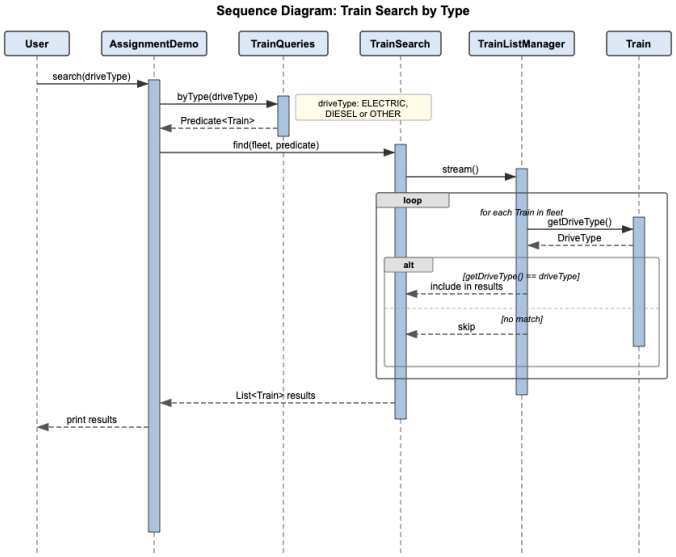
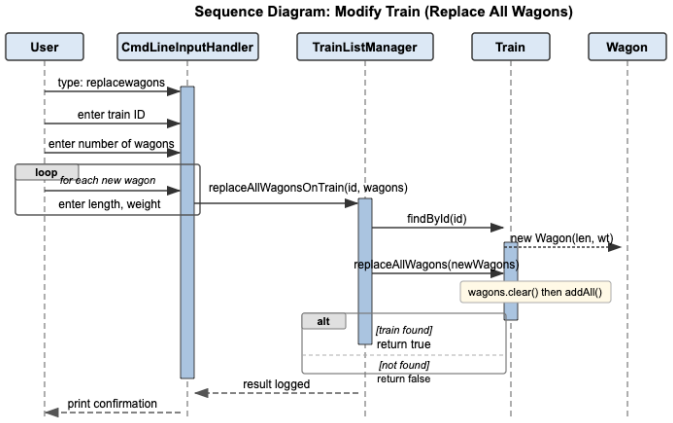
ASSIGNMENTS – OOADI

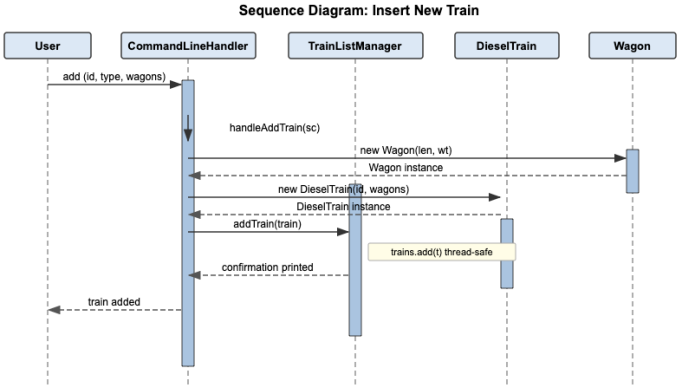
LECTURE 8

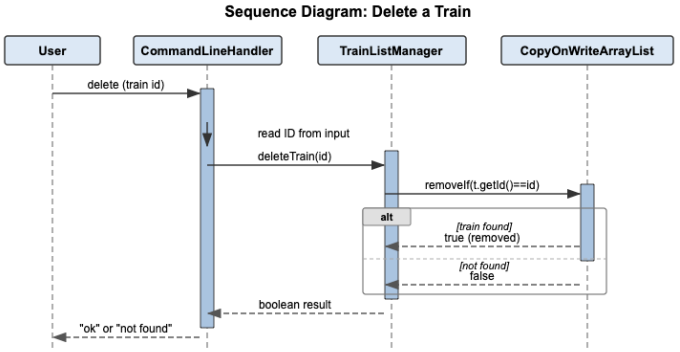
1. To keep it simple I will do it for only for the search by type case.



2. Message sequence diagrams for the new requirements could be as follows.



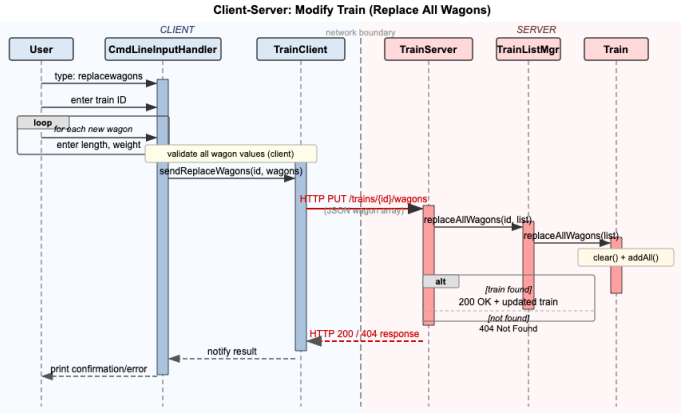


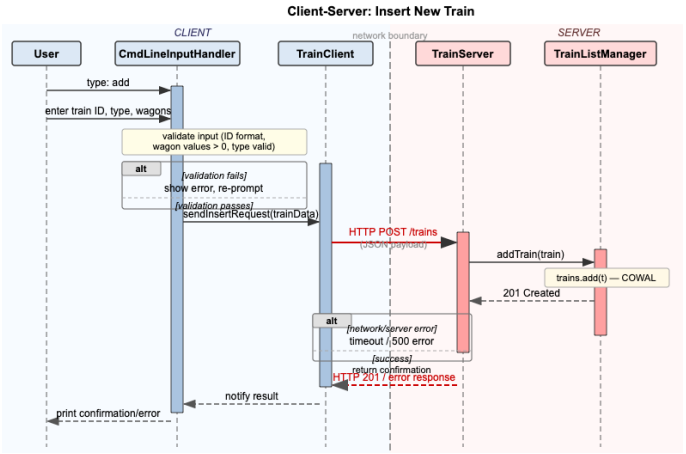


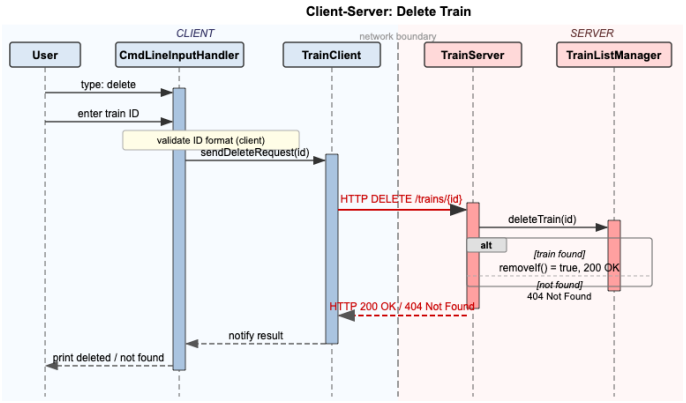
To implement these new functionalities, we need to add new wagon mutation methods to Train.java. Similarly, we need to add new public methods that delegate to the train's mutation methods. We also need to make similar adjustments to CommandLineInputHandler and the TrainOperator. Please look at Moodle for suggested updates to the code base for this.

3. For this a few conceptual changes must be made. For one, the CommandLineInputHandler should stay on the client side, while TrainListManager and the entire train list is to be moved to the server side. This also implies that the direct method calls, from before, between them now become network messages. Input validation should happen on the client before sending. The server needs to send back responses over the network. Failures can now happen that did not exist before, such as network errors and timeouts.

   Compared to the previous diagrams then we need to introduce TrainClient and TrainServer just as we need to illustrate the network boundary. All validation happens on the client side before anything is sent across the network. This is shown as a yellow note box. This is important because sending bad data over the network only to have it rejected wastes a round trip. On the client side we have TrainClient to interface between the input handler and the raw HTTP calls. And in a similar way we have the TrainServer to do the same on the server side. Other than that, most everything is unchanged.

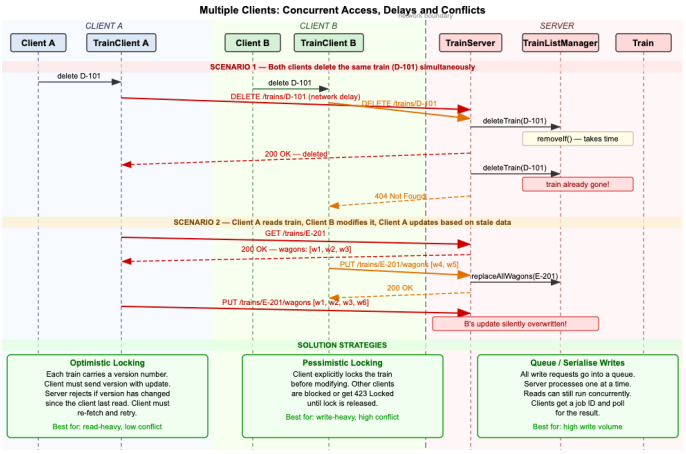






As mentioned, introducing the networking aspect, opens for new failure modes appear that was not present in the local version. The alt fragments now include network timeouts and HTTP 500 errors alongside the original "not found" case. In a real system these need to be handled explicitly, something a local method call never had to worry about.

4. This serves to illustrate the concurrency problems that may arise when working with distributed systems. Two obvious problems that can arise are illustrated in the diagram below.



The two scenarios relate to simultaneous delete operation on the same train and reading of obsolete (old, dated and incorrect) data followed by a subsequent overwrite. Included in the diagram are three possible approaches to solution implementation.

- Optimistic locking adds a version number to every train. The server rejects any update where the version doesn't match what it currently holds — forcing the client to re-fetch and retry. This has low-overhead and works well when conflicts are rare.
- Pessimistic locking lets a client explicitly lock a train before editing it. Other clients are blocked until the lock is released. This is safer but slower and if a client crashes while holding the lock, you need a timeout mechanism to release it.
- Queue-based serialisation sends all writes through a queue and processes them one at a time. This eliminates conflicts but also adds latency. The clients get a job ID and must poll for the result rather than getting an immediate response.